

6. System Design and Architecture

Following the model-development phase, the project progressed from standalone NLP experiments to an integrated phishing email detection system. The resulting architecture was designed around a separation between user interaction, backend inference, machine-learning classification, supplementary security information and explainability. The project contains two genuine explainability implementations built around the same gated-hybrid DeBERTa-v3-large classification approach: a SHAP implementation and a LIME implementation.

The SHAP implementation consists of the DeBERTa-based classification pipeline, FastAPI backend, SHAP explainability component, Streamlit application and browser extension. The model and explanation logic are centralised in the backend rather than independently implemented inside each user interface. Streamlit and the browser extension therefore act as clients of the same prediction service. SHAP explanation is requested separately from ordinary classification because explanation generation requires substantially more computation than obtaining a prediction.

The LIME implementation uses the same hybrid classification concept but is exposed through a Flask application and Chrome extension/API workflow. The Flask service validates the incoming request, invokes the predictor and returns the prediction result as a JSON response. LIME is implemented as the local explanation method and produces influential terms for the selected class. The prediction logic remains in the backend, preventing duplication of the model in the JavaScript side.

Although the two implementations use different backend and explanation configurations, the central classification architecture remains the same. Email content is preprocessed and passed through two information paths: DeBERTa-v3-large provides the contextual representation, while 12 engineered security and linguistic features provide the numerical representation. The two branches are combined through gated fusion before the final Safe/Phishing classification. SHAP and LIME are positioned after classification and explain the resulting model prediction rather than independently deciding whether an email is phishing.

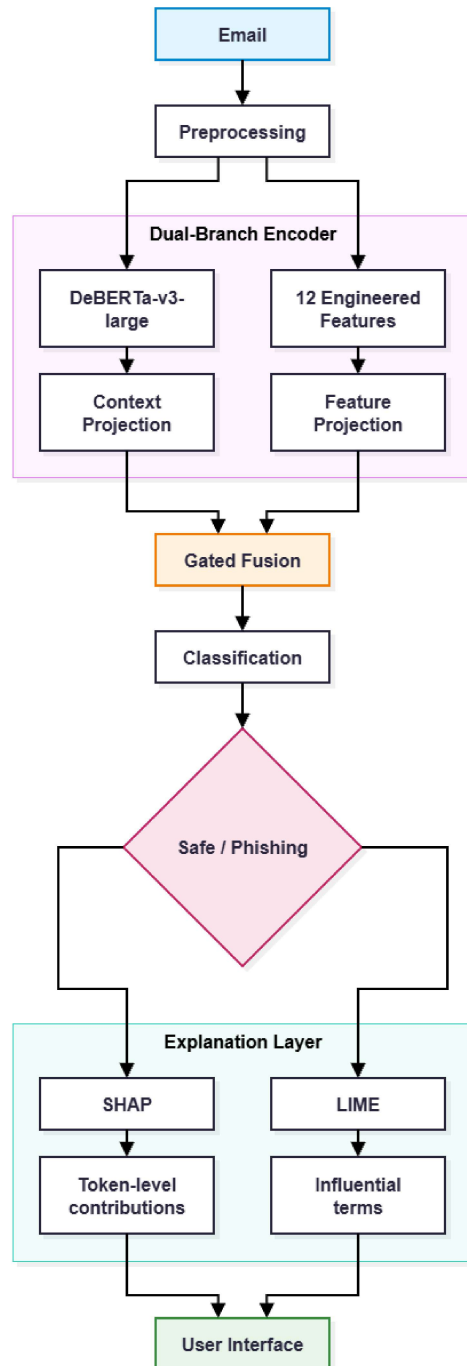


Figure 6.1: Overall System Architecture of the Phishing Email Detection and Explainability System

Figure 6.1 presents the common system architecture used in the project. The email is first preprocessed before entering the dual-branch encoder. DeBERTa-v3-large captures contextual language information, while the 12 engineered features capture measurable linguistic and security characteristics. After context and feature projection, the representations are combined through gated fusion and passed to the classifier. The

resulting Safe/Phishing prediction can then be explained through SHAP or LIME. SHAP provides token-level contribution information, whereas LIME provides locally influential terms and signed weights. The explanation layer therefore supports interpretation of the classifier output but does not replace the classifier.

6.1 SHAP-Based System Design and Architecture

The SHAP-based implementation was developed as one of the two explainability configurations of the phishing-email detection system. It integrates the trained gated-hybrid DeBERTa-v3-large classifier with a FastAPI backend, SHAP explanation pipeline, Streamlit application and browser extension. Within this implementation, the trained classifier remains responsible for producing the Safe/Phishing prediction, while SHAP operates as a post-hoc explanation mechanism for examining the contribution of individual tokens to that prediction.

A central design decision in the SHAP implementation was to separate normal classification from explanation generation. The FastAPI backend provides the common service boundary between the trained model and the user-facing applications, allowing both Streamlit and the browser extension to access the same classification pipeline. Prediction and explanation are exposed separately so that a user can receive the classification result without automatically executing the more computationally expensive SHAP procedure. This separation also keeps the machine-learning and explanation logic out of the user interfaces and avoids maintaining independent copies of the classifier across different clients.

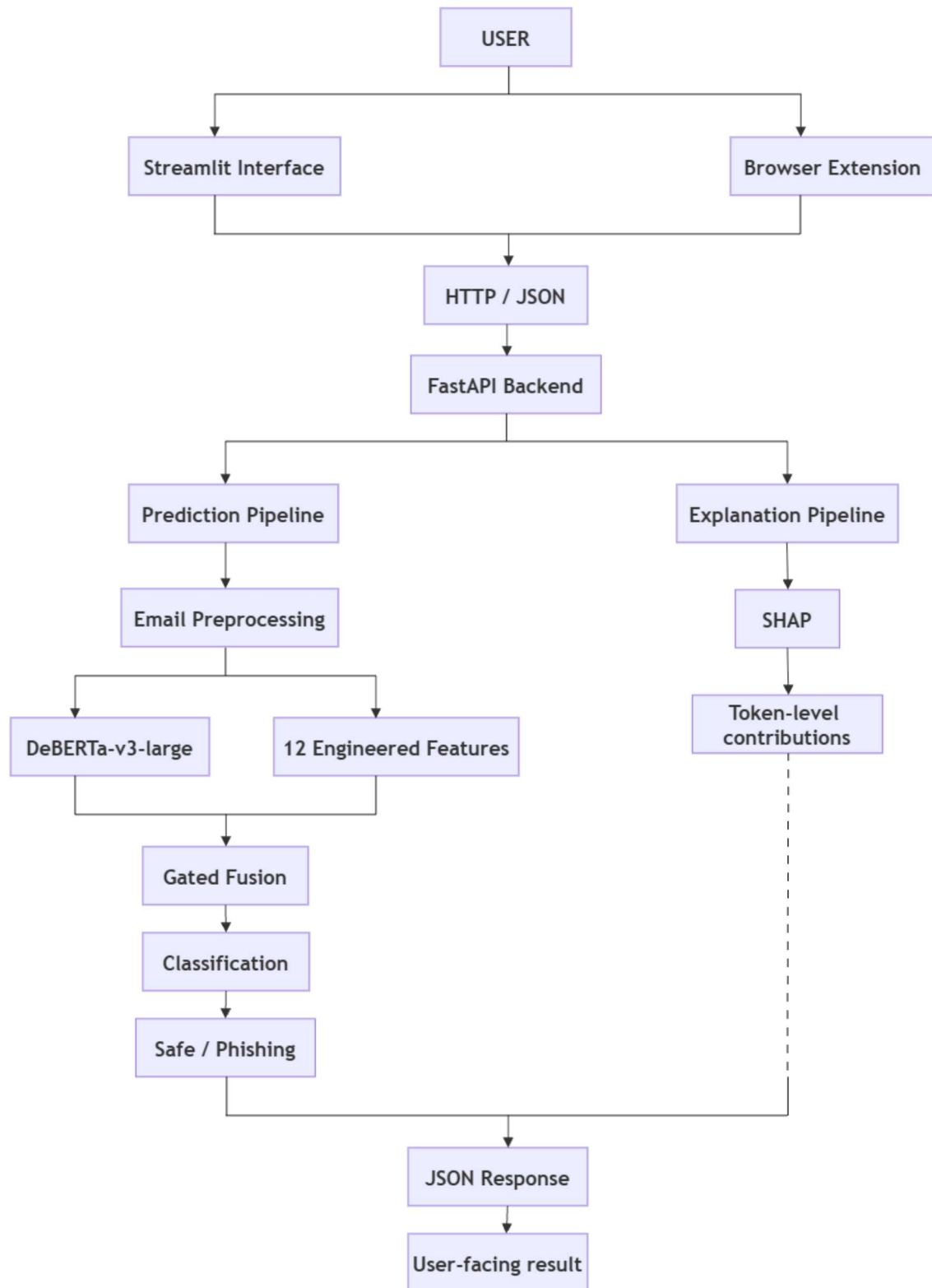


Figure 6.2: Overall SHAP-Based System Design and Architecture

The architecture shown in Figure 6.2 illustrates how the components of the SHAP implementation interact. Email information collected through Streamlit or the browser extension is transmitted to the FastAPI backend through HTTP/JSON. The prediction pipeline preprocesses the email and passes it through the DeBERTa-v3-large and engineered-feature branches before gated fusion and classification. When an explanation is requested, the SHAP pipeline operates on the trained classifier and returns token-level contribution information. Both the prediction and explanation results are then presented through the requesting user interface.

6.1.1 Architectural Design Principles

Several design principles guided the integrated system.

The first was separation of concerns. User-interface code is responsible primarily for collecting or obtaining email information and presenting returned results, while the backend is responsible for model inference. Explainability is additionally separated from ordinary prediction because generating SHAP values is substantially more computationally expensive than obtaining a classification.

The second was interface independence. A backend API provides a common boundary between the classifier and user-facing applications. This enables the Streamlit application and browser extension to use the same classification pipeline rather than embedding separate copies of the model.

The third was prediction/explanation separation. The trained model determines the Safe/Phishing classification. Security indicators and SHAP values provide contextual or explanatory information but do not independently override the model's decision. This distinction is particularly important for a security application because users could otherwise interpret a highlighted keyword or URL as the actual decision rule.

The fourth was progressive system integration. Components were developed and tested incrementally rather than constructing the entire system simultaneously. The project evidence records iterative integration and troubleshooting across the model, API, frontend and explainability components.

6.1.2 Classification Architecture

The principal classification component uses DeBERTa-v3-large together with the engineered-feature/gated-fusion pipeline developed during the modelling stage.

For a submitted email, the system processes the textual content through two information paths. The contextual branch tokenizes the input and passes it through DeBERTa-v3-large to obtain a learned representation of the message. In parallel, the

feature branch calculates the 12 linguistic and structural features. The two representations are projected into compatible learned spaces and combined using the gated-fusion mechanism before the final classification layer produces the Safe/Phishing output.

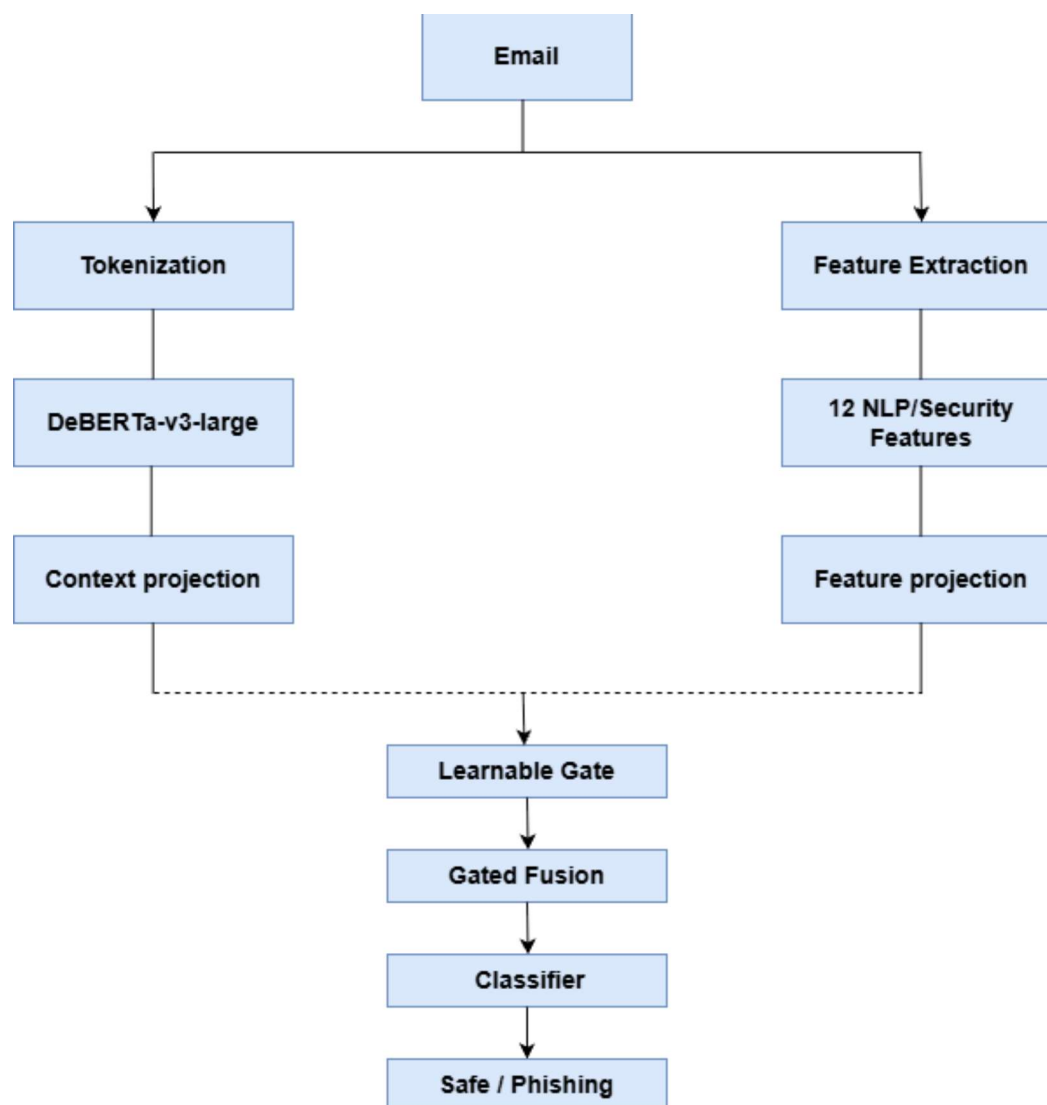


Figure 6.3 Gated Hybrid DeBERTa model

The motivation for retaining both branches was not that explicit indicators were assumed to be intrinsically more reliable than contextual NLP. Instead, the hybrid experiment investigated whether interpretable linguistic and structural characteristics could complement the contextual representation learned by DeBERTa.

This distinction also affects system presentation. For example, the existence of a URL, phone number or email address is treated as a structural characteristic rather than automatically as evidence of phishing. The browser-extension implementation explicitly separates these neutral characteristics from actual phishing indicators.

This avoids a simplistic rule such as: *URL present* $\Rightarrow$ *phishing* ; which would be unsuitable because legitimate emails routinely contain links and contact details.

6.1.3 FastAPI Backend Design

FastAPI acts as the integration layer between the machine-learning components and the user-facing interfaces. Rather than requiring Streamlit or the browser extension to load and execute the transformer directly, the frontend submits email information to the backend and receives structured results.

This design provides an important engineering abstraction:

Frontend $\rightarrow$ HTTP request $\rightarrow$ FastAPI $\rightarrow$ ML inference $\rightarrow$ JSON response

The system exposes prediction and explanation functionality separately. The ordinary prediction workflow is provided through the `/predict` route, while explainability is handled through `/explain`.

The input payload used by the interfaces contains: sender, subject, body

The Streamlit implementation constructs this payload and sends it to the prediction endpoint as JSON. The browser extension follows the same basic communication pattern, submitting the extracted sender, subject and body to the backend.

Table 6.1: Principal API Responsibilities

Component	Responsibility
<code>/predict</code>	Execute normal classification workflow
<code>/explain</code>	Generate prediction-associated SHAP explanation
Input	Sender, subject and email body
Model layer	DeBERTa-based hybrid classification
Output	Prediction information and associated response fields
Client applications	Streamlit and browser extension

Separating `/predict` and `/explain` is preferable to automatically running SHAP for every classification. Normal prediction can be returned without waiting for the substantially heavier explanation procedure, while an explanation can be generated when required.

6.1.4 Streamlit Interface Design

Streamlit provides the primary standalone user-facing prototype. It allows a user to enter email information and submit it for analysis without directly interacting with the model or API.

The frontend sends the following JSON structure to FastAPI:

```
{  
  sender: <sender>,  
  subject: <subject>,  
  body:  <email body>  
}
```

The source implementation confirms that normal prediction is sent to the prediction endpoint using an HTTP POST request. The interface also implements explicit handling for connection failures, request timeouts and other API errors.

The operational sequence is therefore:

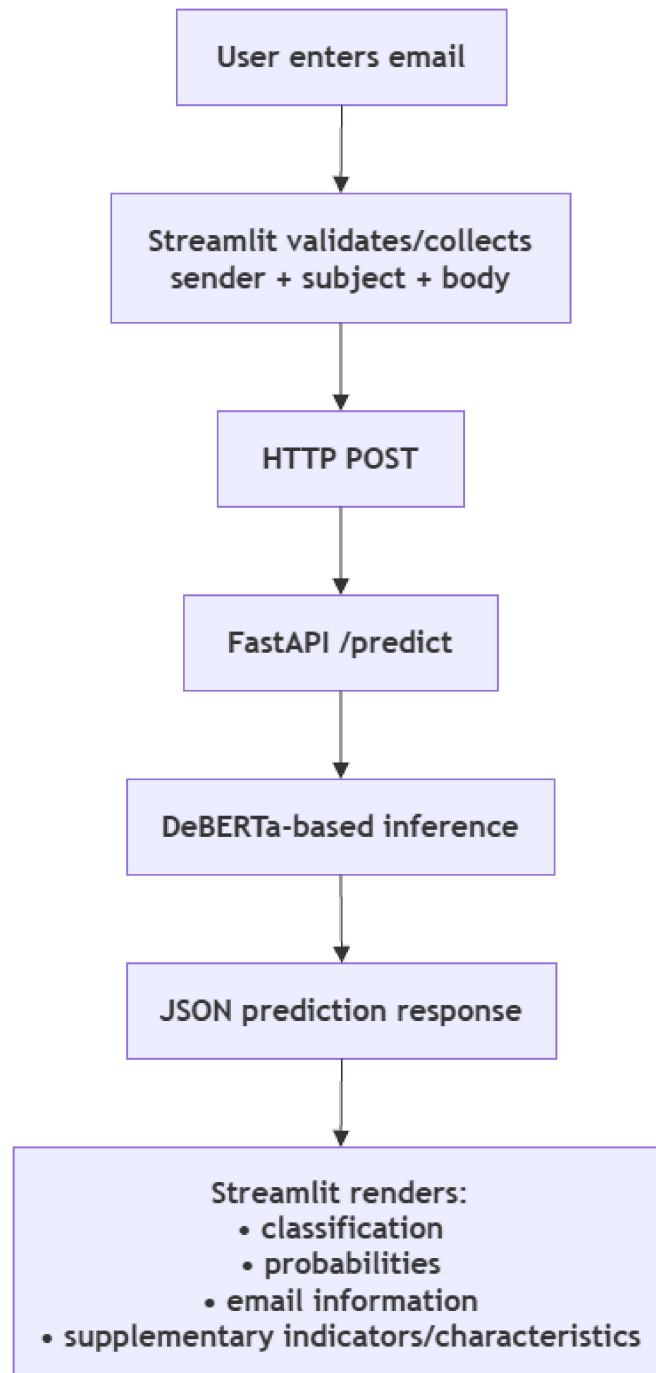


Figure 6.4: Streamlit prediction workflow

Explainability is requested separately:

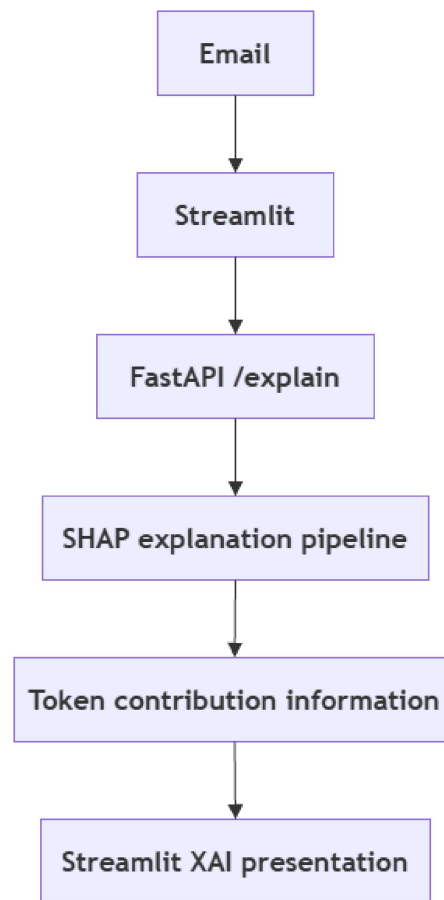


Figure 6.5: Explainability Request

The implementation therefore avoids making SHAP generation a prerequisite for receiving the initial classification. This became particularly important after the project encountered substantial computational overhead during SHAP generation.

The Streamlit client also distinguishes security indicators from neutral email characteristics. The implementation explicitly states that the presence of a URL, email address or phone number is not by itself considered a phishing indicator.

This is a useful usability decision because presenting every common structural property as suspicious could produce misleading explanations and reduce user trust.

6.1.5 Explainability Architecture

The explainability layer was implemented using SHAP as a post-hoc mechanism. Its architectural position is intentionally downstream from the trained classifier.

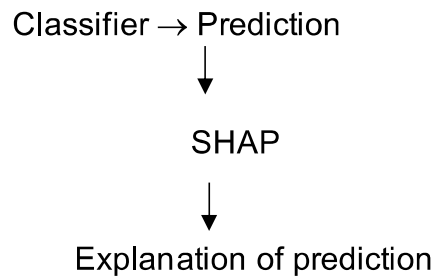
The relationship can be represented as:

$$f(x) \rightarrow \hat{y}$$

where f is the trained classifier, x is the email and $\hat{y}$ is the model prediction.

SHAP then analyses the behaviour of f around that prediction to estimate token-level contributions. It does not produce an independent ground-truth judgement.

Therefore:



rather than: SHAP → decides whether email is phishing

This distinction is essential for interpreting the resulting interface. A token with a strong SHAP contribution indicates that the token influenced the model output under the explanation procedure; it does not prove that the word is inherently a phishing indicator.

6.1.6 SHAP Computational Refinement

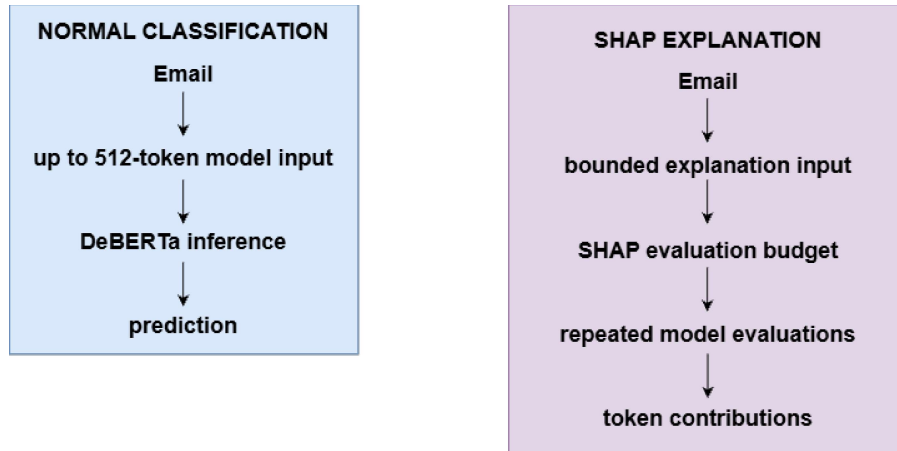
The original explanation implementation introduced an important engineering challenge. Applying SHAP to DeBERTa-v3-large required repeated model evaluations, which resulted in long execution times and session instability. The project therefore separated the computational requirements of classification from explanation generation rather than reducing the normal classifier's input configuration.

The revised SHAP implementation introduced bounded explanation parameters:

$$\begin{aligned} SHAP_MAX_TOKENS &= 64 \\ SHAP_MAX_EVALS &= 150 \\ SHAP_BATCH_SIZE &= 4 \end{aligned}$$

The explanation route also performs garbage collection and clears the CUDA cache where available before generating the explanation. The shortened text is used specifically for SHAP processing, after which the explainer executes with explicit `max_evals` and `batch_size` constraints.

Consequently, two different computational paths exist:



This represents a deliberate engineering trade-off. Restricting the explanation workload improves practical stability and responsiveness, but the explanation may not represent every token considered by the full classification path.

6.1.7 Browser Extension Design

The browser extension was implemented as a stretch goal because a standalone web application requires the user to leave the email-reading context, copy message content and paste it into another interface. For intended users such as students, staff or ordinary email recipients, that additional friction can reduce practical use. A browser extension allows analysis to be initiated while the user is viewing the message.

The extension uses Manifest V3. The manifest defines the name University Phishing Email Detector, permissions activeTab and scripting, host permission for <http://127.0.0.1:8000> , a popup action and a content script. The content script is configured for localhost/127.0.0.1 test pages and Gmail.

The extension has two related interaction paths in the supplied implementation. The popup script obtains the active browser tab, sends a {action:'getEmail'} message to content.js, validates that a body was returned and POSTs sender, subject and body to the /predict endpoint. The revised content script also supports an injected in page 'Scan Email for Phishing' button in the controlled dataset test environment and can call /predict directly.

In the controlled test environment, content.js searches visible h1/h2/h3 elements for a heading equal to 'University Email'. It then traverses up to eight parent elements until it finds a visible container with sufficient text and 'From:' and 'Subject:' metadata. Sender and subject are extracted from visible p/div/span/strong/b elements whose text begins with the corresponding label. The body is extracted from a cloned email card after headings/header/ground-truth elements are removed, and metadata lines such as 'University Email', 'From:', 'Subject:' and 'Dataset ground truth:' are filtered out.

This extraction logic is intentionally conservative for the controlled test page. Although the manifest also matches Gmail, the surfaced revised extraction function is specifically oriented to the project's 'University Email' test-card structure. Therefore, the final report should not claim robust Gmail DOM extraction unless a separate Gmail-specific path is verified. This is a significant deployment limitation because webmail DOM structures can change.

After extraction, the extension serialises the email as JSON and sends an HTTP POST with Content-Type application/json to `http://127.0.0.1:8000/predict`. The backend performs exactly the same preprocessing and model inference used by Streamlit. This reuse is important: the extension does not contain a second model and does not reimplement the classifier in JavaScript.

The prediction response is parsed and displayed in an overlay/popup. The revised UI distinguishes phishing predictions from safe predictions, shows confidence and probabilities, and prevents a phishing prediction from being accompanied by the reassuring phrase 'No common phishing indicators detected'. This refinement addressed a usability problem in which rule-based indicators could otherwise contradict the learned model's verdict.

The content script also supports on-demand XAI. When the user selects 'Explain Decision', it POSTs the same sender/subject/body to `/explain`. The returned explanation is displayed as factors supporting the prediction and factors working against it. This keeps XAI optional, which is appropriate because explanation is slower than classification.

Error handling is implemented at several layers. The popup checks that an active tab exists, that `content.js` returns success and a non-empty body, that the HTTP response is successful and that the API does not return `success=false`. The content script similarly catches extraction/API errors, displays a visible warning, disables buttons during requests and restores them afterward. These controls improve usability because backend or extraction failure is communicated explicitly rather than appearing as a misleading 'Safe' result.

From a privacy perspective, the extension can access email text on matched pages and sends that text to the local backend. The current host permission is restricted to the local API, which is preferable to broad remote-host permission, but content-script access still needs to be justified and minimised. A production extension would require a clear privacy notice, secure backend authentication, HTTPS for remote deployment, careful permission review and explicit retention/logging controls.

From a usability perspective, the extension places the security decision close to the user's reading context and reduces manual copying. However, browser popups and overlays have limited space, SHAP output can be verbose, and users may over-trust a

high confidence score. The interface should therefore prioritise the verdict, probabilities and concise explanation while keeping detailed attribution optional.

There is no meaningful 'browser-extension classification equation' because the extension is an orchestration and presentation layer rather than a statistical model. Mathematically, its role can be represented as a composition of functions:

$$\text{Extension output} = \text{UI}(\text{API}(\text{Extract}(\text{page})))$$

where $\text{Extract}(\text{page})$ produces sender/subject/body, API invokes the trained classifier, and UI renders the returned verdict. The actual class probability is still produced by the backend softmax model, not by JavaScript.

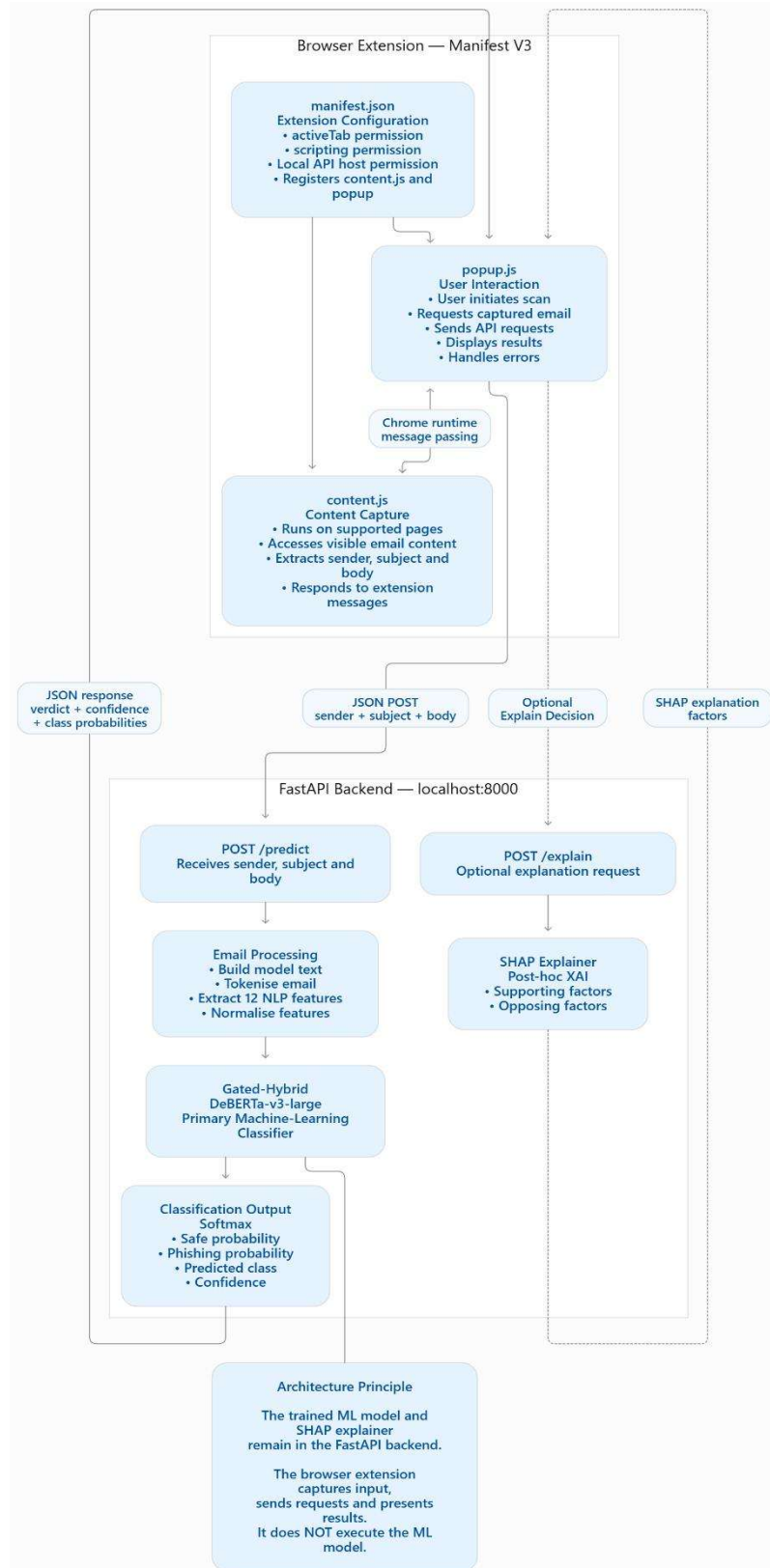


Figure 6.6: Browser Extension Component Architecture

The Manifest V3 browser extension separates webpage interaction from machine-learning inference. The extension configuration is defined by `manifest.json`, while `content.js` captures email content from supported pages and `popup.js` coordinates user interaction and communication with the backend. The extracted sender, subject and body are transmitted as JSON to the FastAPI `/predict` endpoint, where preprocessing and gated-hybrid DeBERTa-v3-large inference are performed. SHAP explanations are requested separately through `/explain`. The trained model and XAI implementation remain centralised in FastAPI; no machine-learning model is executed within the browser extension.

6.1.8 Browser Extension Components

The supplied source establishes at least three important extension responsibilities: the popup interface, popup control logic and page/email extraction through a content-script communication workflow.

When the user initiates analysis, `popup.js` queries the currently active browser tab: `chrome.tabs.query(...)`. It then sends a message to the tab requesting the email: `action: "getEmail"`

The returned object is expected to contain detected email information, including the body and where available, sender and subject. The extension checks whether an email was successfully obtained before continuing. The popup displays classification information together with email details and supplementary indicators. The source also includes logic to display Safe and Phishing probabilities and normalize their presentation as percentages.

The resulting workflow is:

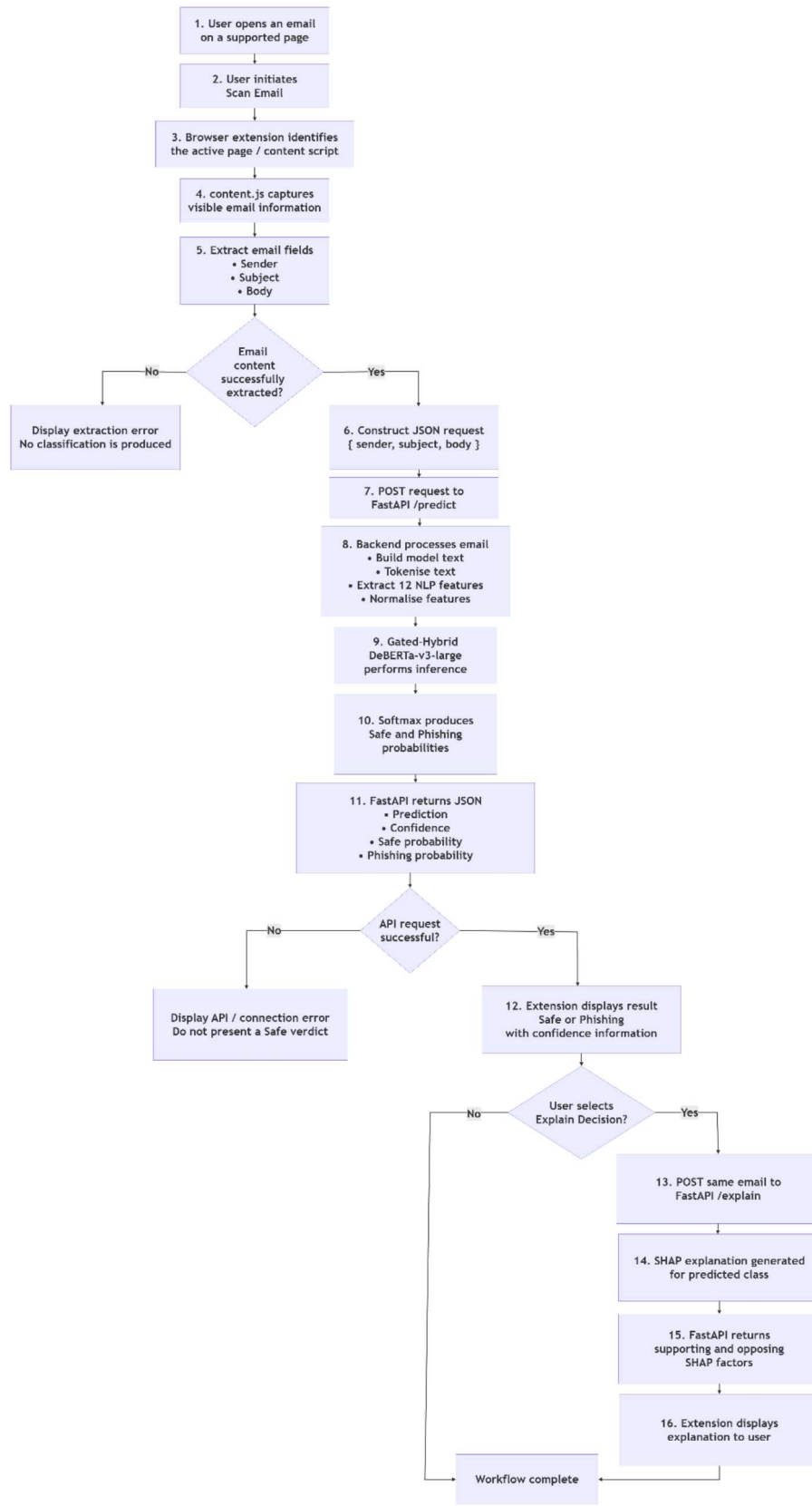


Figure 6.7: Browser Extension Email Capture Workflow

The workflow begins when the user initiates a scan on a supported email page. The content script extracts the available sender, subject and body, which are transmitted as JSON to the FastAPI /predict endpoint. Classification is performed centrally by the gated-hybrid DeBERTa-v3-large model. The returned verdict and confidence information are displayed by the extension. SHAP explanation is performed separately and only when the user requests Explain Decision.

6.1.9 Browser Extension User Interface

The extension popup was designed to provide information directly in the browsing context rather than redirecting the user to another application.

The interface contains email information such as sender and subject and includes an area for phishing indicators. The supplied final popup HTML identifies the system as powered by DeBERTa-v3-large.

The extension logic further distinguishes four presentation situations:

- 1) safe prediction with no phishing indicators;
- 2) safe prediction with neutral characteristics;
- 3) phishing prediction with supplementary indicators;
- 4) phishing prediction where no rule-based indicator is available.

This distinction is explicitly documented in the source code.

Of particular importance, the source contains a safeguard stating that a phishing prediction must not display the reassuring message:

“No common phishing indicators detected”

as though that contradicted the classifier.

This is a small but meaningful usability decision. A model can classify an email as phishing even when simple manually recognizable indicators are absent. Displaying a green “no indicators” message in that situation could create contradictory risk communication.

6.1.10 Supplementary Indicators versus Model Prediction

A central design principle across the system is that supplementary indicators do not determine the model prediction. The DeBERTa-based model remains the primary classifier. Rule-based information is used to communicate additional context to the user. This produces three conceptually different forms of information:

Information	Purpose	Determines classification?
Model prediction	Safe/Phishing classification	Yes
SHAP contribution	Explain model behaviour	No
Supplementary security indicator	Provide understandable contextual information	No

Maintaining this distinction avoids presenting the system as a keyword-based phishing detector when its classification actually originates from the learned model.

It also helps prevent another interpretability error: an email may contain suspicious-looking terms while still being classified safe, or may be classified phishing without containing one of the manually specified indicator patterns. The contextual classifier and supplementary indicators therefore answer different questions.

6.1.11 Error Handling and System Robustness

A usable ML prototype must address failures outside the classifier itself. The implemented interfaces therefore contain error-handling mechanisms for communication and extraction failures.

For Streamlit, the source handles:

- inability to connect to the API;
- request timeout;
- general HTTP/request errors; and
- separate timeout handling for SHAP explanation.

For the browser extension, the source checks whether an active browser tab can be accessed and whether valid email content has been returned before sending the classification request. If email extraction fails, an error is raised rather than sending an empty message to the classifier.

These controls do not make the prototype production-ready, but they improve robustness during demonstration and functional testing by distinguishing model output from infrastructure failures.

6.1.12 Usability Considerations

The system provides two interaction modes with different purposes.

Streamlit is useful as a controlled standalone environment for entering test messages, inspecting classification outputs and requesting detailed explanations. This makes it particularly suitable for development, demonstration and analysis.

The browser extension reduces interaction overhead by placing classification closer to the email being viewed. The user initiates scanning from the extension and receives the classification without manually copying the complete email into another application.

The extension therefore addresses a usability limitation of the standalone prototype:
manual transfer of email → in-context analysis

However, convenience should not be confused with automatic protection. The current prototype provides decision support to the user; it is not documented as an email gateway that automatically blocks messages or prevents malicious actions.

This distinction is important both technically and ethically.

6.1.13 Deployment Considerations

The implemented system demonstrates functional integration but remains a prototype architecture.

The local FastAPI configuration is suitable for controlled development and demonstration but would require significant engineering before public deployment. A production architecture would need to consider authentication, HTTPS, concurrency, model-serving infrastructure, resource management, monitoring, rate limiting and secure handling of email data.

DeBERTa-v3-large also introduces computational deployment costs. Its strong predictive performance must therefore be considered alongside memory consumption and inference latency. SHAP is still more computationally expensive because explanations require repeated model evaluations, which is why the project introduced a bounded explanation pathway.

A production design could therefore retain the architectural separation already demonstrated in the prototype:

Fast prediction service

- Immediate classification
- Optional explanation service

6.1.14 End-to-End System Workflow

Combining the preceding components gives the complete implemented workflow.

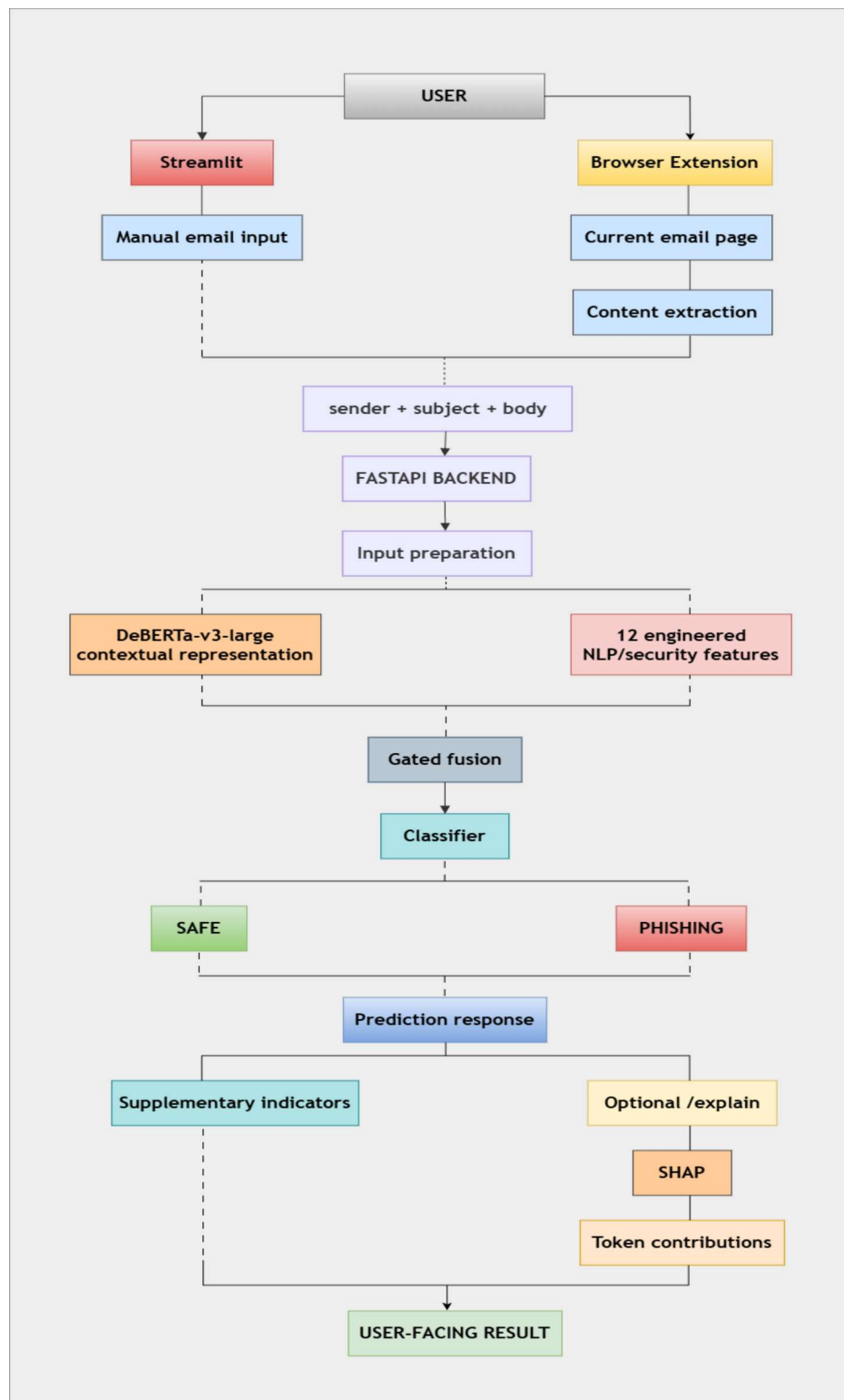


Figure 6.8: End-to-end phishing detection architecture

The architecture demonstrates that the project extends beyond training an NLP classifier. The final prototype connects data preparation, contextual language modelling, engineered feature extraction, learned feature fusion, API-based model serving, post-hoc explainability and two user-facing interaction mechanisms.

At the same time, the architecture deliberately preserves boundaries between these components. The classifier predicts, SHAP explains the classifier, supplementary indicators provide additional context, FastAPI exposes the analytical services and Streamlit and the browser extension present those services to the user. This separation provides a clearer basis for evaluating both predictive performance and system behavior independently.

6.2 LIME-Based System Design and Architecture

The project also retains the LIME-based implementation as a separate prototype configuration. The phishing-email detection workflow was implemented in a multi-layered web application separating the user experience, request routing, machine-learning inference, explainability, and model persistence layers. The prototype is a combination of transformer-based language model and twelve engineered security and linguistic features. Implementation is done using the Local Interpretable Model-agnostic Explanations method, using LimeTextExplainer.

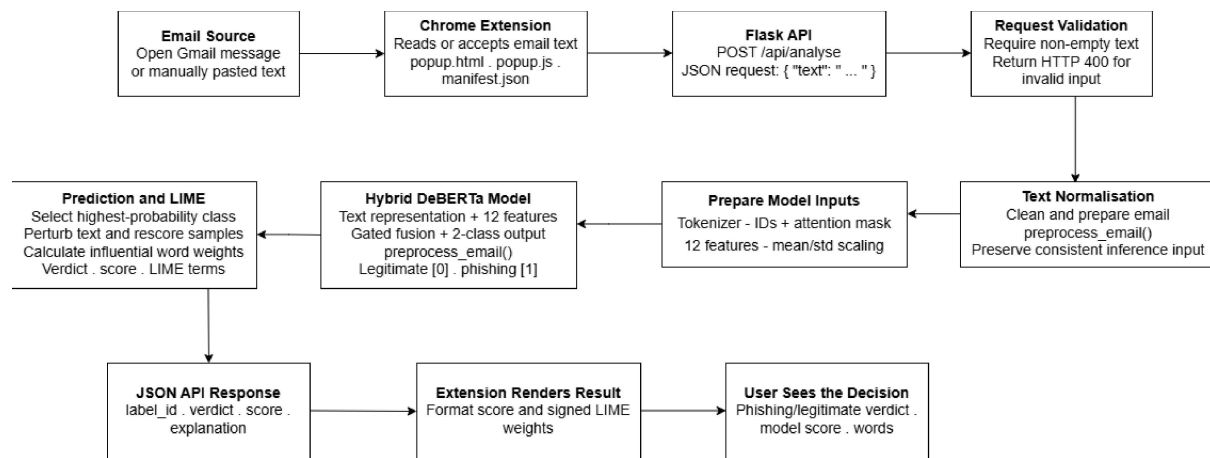


Figure 6.9: LIME-Based System Design and Architecture

The presentation layer includes a Chrome extension popup and a different command line API client. The extension can obtain the visible text of an open Gmail message using a content-extraction script. This is in addition to a manual text-entry option as automatic extraction may be impacted by changes to the document structure in Gmail. The manually entered/extracted information is passed to the back end as a JSON request.

The Flask application (app.py) is the service layer. It unmarks the health-check route and the email-analysis route, validates the incoming requests, invokes the predictor and returns the prediction result as a JSON response. API has no implementation specific logic in it. This helps to isolate the machine learning responsibilities from the web-service responsibilities and enables the same predictor to be used by an evaluation script, by the browser extension or by another authorised client.

The prediction layer is implemented in predictor.py. Loads the tokenizer, transformer configuration, trained weights, engineered-feature statistics and model settings from the model directory. These resources are loaded only once, at start-up of Flask, and not for every request. This design is crucial since the backbone of DeBERTa-v3-large is computation-intensive while running inference on a CPU machine.

The trained classifier has a hybrid architecture. The contextual representation generated by the transformer is projected to compatible representations along with the 12-dimensional engineered-feature vector before gated fusion and binary classification. The 12-dimensional engineered-feature vector and the contextual representation generated by the transformer are transformed to compatible representations before gated fusion and binary classification. This design aligns with the findings that both contextual language modelling and the explicit identification of social-engineering behavior (e.g., phishing) are effective techniques for detecting phishing (Al-Subaiey et al. 2024; Uddin, Mahiuddin & Sarker 2026).

A request consequently travels through the architecture one way as an email message and the other way as a decision in structure:

Validation in Chrome extension or API call, preprocess (Flask), tokenisation/feature extraction, hybrid classifier, LIME explanation, JSON response, UI.

The components do not share user-interface state directly, but rather through clearly defined interfaces. This separation helps to facilitate future development and reproducibility. If later a web interface is developed instead of the local Chrome extension, the model implementation does not need to be modified. Likewise, the model could be used in the background of the existing production web server, with the same requests and responses.

6.2.1 Email Processing Pipeline

The email-processing pipeline takes an unstructured email body and converts it to a prediction and human-readable local explanation. This appears to be one analysis to the user, but it is several different stages as far as the backend is concerned.

First the extension reads the visible body of the email from Gmail, or the user pastes text. The client has a preliminary test for whether the request is empty, applies the max length of input before he/she generates the request. It is then sent to:

POST /api/analyse

The request body is provided in the following format:

```
{  
  The following text will be analysed: "text": "Email content to be analysed"  
}
```

The extension does not need to validate the request; it's done independently by Flask. This is because the endpoint can be directly called from a PowerShell, Python or other HTTP client. The backend validates that the payload is a JSON object, the text field is a string, the string is not empty after removing leading/trailing whitespace, and the text field

is no longer than 30,000 characters. Requests with invalid data are not passed to the model and instead are returned with an appropriate "client-error" response.

The accepted text is then preprocessed in the same manner as it was by the deployed classifier. Preprocessing processes removes HTML tags, control characters, reduces repeated white space and decode HTML character references, and also performs Unicode NFKC normalisation. The reason Unicode normalisation is relevant is because of the fact that visually similar characters may be represented in different ways, and the inconsistent normalisation can result in unstable tokenisation. When markup is removed, common HTML formatting also cannot be interpreted as regular email.

The pipeline after pre-processing produces two model inputs concurrently. The first argument is the sequence of transformer tokens (token ids and attention mask). The number of messages in any sequence is limited to the maximum sequence length defined in the model configuration, which results in long messages being cut off. The second is a vector of 12 numerical features extracted from the same text after cleaning. Ensuring the same normalised content in both branches eliminates the mismatches in data from the transformer and feature extractor.

Standardisation of the numerical vector is performed using the feature means and standard deviations saved while training the model:

$$z_i = \frac{x_i - \mu_i}{\sigma_i}$$

where x_i is the observed feature value, (μ_i) is the mean of the training set for the feature, and (σ_i) is the standard deviation of the training set for the feature. Re-using the training statistics is crucial since computing them from each email is bound to change the scale of the input to the model and might cause the learned weights to be invalidated.

Transformer tokens and the standardised numerical features are fed into a hybrid model. The model generates two logits, one for the correct class and one for the fake (phishing) class. We use a softmax function to transform the logits into probabilities for each class. The most likely class is the verdict. Label 0 means a legitimate e-mail, and label 1 means a phishing e-mail in the deployed implementation.

If an explanation is asked for, LIME produces modified versions of the email, conceals or omits certain terms, and continues to ask the classifier. It subsequently fits a simpler local model around the current message and estimates which are terms that affected the selected class. Lastly, Flask merges the verdict, selected-class probability and LIME terms into a single JSON response.

This pipeline guarantees that the same pre-processing and classifier is applied by the Chrome extension as in the evaluation process. It is helpful to note, however, that there is a difference between functional testing and model evaluation. The correct JSON response is a sign that integration is correct; it does not represent generalisation performance. The precision, recall, F1 score and confusion matrix need to be computed for some labelled emails that were not included in the training of the model (Kyaw, Gutierrez & Ghobakhlou 2024).

6.2.2 NLP Detection Component

Utilizing a gated hybrid model (DeBERTa-v3-large). DeBERTa separately encodes word content and word position to enhance the representation of words in context. DeBERTa-v3 further introduces replaced-token detection objective (as in ELECTRA) and gradient-disentangled embedding sharing (He, Gao & Chen 2023) into the pre-training process. The features contribute to its effectiveness as a contextual model for discriminating messages which might use the same words, but with different intent.

For instance, the term “password” can be included in legitimate and spam e-mails. This could include a phishing email asking you to confirm a password using a link in the email, or a security-awareness message telling you that you should never enter a password using a link in an email message. Both messages can be returned because of a keyword-only system. The other branch of the transformer considers the surrounding language, relationships between words.

The contextual output has a hidden representation of 1,024 dimensions. It is believed that this representation would be projected to 256 dimensions in the implemented architecture. The 12 engineered features are each projected to another 256 dimensional representation. The two projections are combined into a 512 dimensional vector, and fed into a sigmoid gating layer. The gate determines the amount of information to save from the contextual and engineered branches, before using the fused representation for delivering to the final two-classifier.

The numerical branch is used to compute the following 12 features:

Feature group	Calculated indicator
Social-engineering vocabulary	Counts of urgency, credential, threat, financial/payment and call-to-action terms
Contact and link indicators	Counts of URLs, email addresses and telephone numbers

Writing-style indicators	Numbers of exclamation marks and question marks
Capitalisation	Ratio of fully uppercase words
Message size	Log-transformed message length

The following indicators were selected because they are common in phishing and business-email-compromise (BEC) messages. However, the numerical characteristics must be taken as indicators but not rules. If a payment language appears in an e-mail that is supposed to contain an invoice, then the e-mail is legitimate; likewise, if a URL appears in a security notification meant to be sent to the recipient, then the e-mail is legitimate. In the hybrid model, the model is able to consider these features in combination with surrounding text, rather than relying on one feature to indicate phishing.

The URL feature is currently able to count the number of strings containing a URL, but it does not check the reputation of the domain, expand redirects or do WHOIS or lexical analysis of the hostname. Similarly, the number of email addresses and telephone numbers provide information about the presence, but not about the identity or reputation of the sender. The restrictions should be mentioned explicitly since the present model analyses content of the message instead of performing comprehensive email forensics.

The method `predict()` works for direct inference and returns the class label, verdict, the selected-class score and the two class probabilities. It is also convenient for the purpose of evaluation of the data sets because when it is independent from LIME, an explanation can be generated for each message, thereby creating additional processing efforts. It also guarantees the classification result can be tested without the explanation layer, which is the layer responsible for the explanation of the result.

In the recent research, transformer-based phishing detection has shown to be effective and the achieved accuracy is dependent on the composition of the dataset, class balance, and similarity between training and test data sets (Atawneh & Aljehani 2023; Kyaw, Gutierrez & Ghobakhlou 2024). Therefore, the softmax score deployed should not be reported as a system's overall accuracy. It is the model's confidence in one prediction, and it can be poorly calibrated if the email is not similar to the distribution used to train the model.

6.2.3 Explanation Component

An explanation component has been added to improve the model's user-friendliness. When a legit business mail is misidentified as a security alert, the verdict without supporting information may not be easily accepted in a security application. Explainability can also be used to assist error analysis, determining if this decision has a related social-engineering language or a non-relevant correlation.

Implementation is done using the Local Interpretable Model-agnostic Explanations method, using LimeTextExplainer. LIME is model agnostic and thus does not need to be given access to the internal layers and gradients of the classifier. Instead, it creates perturbed versions of the selected email, observes changes in the classifier's probabilities and fits an interpretable local surrogate model. These yields weighted terms for the decision close to that input.

The class names are set to legitimate and phishing, and the random seed is set to 42 to reproduce the same random numbers, there are 8 terms shown, and 50 perturbed samples. A random seed is used to ensure repeatability when testing. The sample number is relatively small, resulting in limitations in response time in the available CPU hardware, but also stability and fidelity of the system with a larger perturbation set would be lost.

LIME can be called by `predict_with_explanation()`, and the normal predictions can be called by `predict()`. This separation is designed. Evaluation scripts can get predictions efficiently without having to generate explanations, while the API for the user can return a verdict and supporting terms in one go.

The local weights of each of the displayed terms are signed. For the class to be explained:

- if the weight is positive, the term is related to that class;
- a negative weight means this term causes the local surrogate model to move further from this class; and

The larger the absolute weight, the more influence the local data has on the approximation.

Negative does not necessarily indicate that the word is totally safe and positive does not necessarily indicate that the word is malicious. The weights are not percentage or causal weights. They explain the behavior of an approximate local model around a single email. The same term can have a different weight in another message due to changes in the context. This distinction matters as explanation weights are potentially misrepresented in such a way that it could cause false confidence in the model (Salih et al. 2025).

LIME has recently been applied to phishing research to identify positive and negative terms in making the decision and to aid analyst interpretation of the individual phishing classifications (Uddin, Mahiuddin & Sarker 2026). But it is important to evaluate the quality of explanations critically. A plausible explanation does not ensure that the original model is good in terms of sound reasoning. When the random sampling or the number of samples changes, LIME may give different term rankings. However, the extension does not claim to be the model's "reasoning" but simply labels the output "influential terms".

6.2.4 API and Interface

The Flask application provides a small and clearly defined API. The health-check endpoint is:

GET /

It returns a simple confirmation that the phishing API is running. This endpoint is useful during the demonstration because it separates server-availability testing from model inference.

The analysis endpoint is:

POST /api/analyse

A successful response follows this general structure:

```
{
  "label_id": 1,
  "verdict": "phishing",
  "score": 0.9981,
  "explanation": {
    "method": "LIME",
    "class_explained": "phishing",
    "influential_words": [
      {
        "word": "verify",
        "weight": 0.21
      }
    ]
  }
}
```

Label ID, Verdict, and Score are all machine-readable, while the Label ID supplies the numeric class, the Verdict supplies a readable label, and the Score supplies the softmax probability assigned to the selected class. The explanation object specifies the method (LIME) and the locally influential terms and signed weights.

The Chrome extension is a thin client. It gathers text, passes the request back to the backend and shows the result. Prediction logic stays in the backend. This prevents the

duplication of the model in the JavaScript side, and the direct API test and the browser based test use the same classifier.

The extension is based on Chrome Manifest V3 and asks for activeTab, scripting and access to the localhost API. But unlike every open browser tab, activeTab will allow the user to access the page for a limited period of time. Extension permissions are recommendations from Chrome to reduce permissions that could be given to an extension that might be compromised (Google 2024). The present design therefore does not require unrelated permissions like browsing-history, downloads or persistent access to all websites.

Interface shows loading status during analysis, and indicates whether the input is empty, the API service is unreachable, invalid response, request timeout. Because LIME needs multiple predictions from models, it is possible to be slow when DeBERTa-v3-large is running on the CPU, a 120-second timeout is used. This makes it easier for the user to use, but also shows that the current prototype was designed for assessment and experimentation, and not for high-volume real-time mail processing.

6.2.5 Deployment Environment

The prototype is locally deployed on a Windows computer with Visual Studio Code, Python virtual environment and Flask. The virtual environment serves to separate the application's dependencies from the system Python (python). Flask, PyTorch, Transformers, NumPy, SentencePiece, scikit-learn and LIME are required packages. This will aid in reproducing the result on another compatible computer, as these dependencies are recorded in the requirements.txt file.

The application is attached to: <http://127.0.0.1:5000>

The loopback address is a service that is available within the same computer but not exposed on the local network as part of the computer's design. This helps to meet the demonstration requirement without exposing any unnecessary network. When the user uses the service as an ordinary local service, the message text will not be sent to a cloud inference service.

The model is created only when the application is first run (app.py). This generates an obvious start up time but prevents the reloading of a big transformer on each request. Once initialised, Flask stays waiting for requests and will re-use the same model instance. Inference mode (0): It is used to disable calculation of gradient which will help to save memory and computing power during prediction.

The deployment should still be considered a local prototype and not a production service. According to the Flask documentation, the built-in development server is not intended for production deployment because it's not as secure, stable or efficient (Pallets Projects

2025). A production version should deploy an appropriate WSGI server or a containerised version of the service, have controlled network exposure, have a structured log, have authentication and rate limiting and have monitored resource limits.

Containerisation would also ensure deployment uniformity, by encapsulating the Python runtime, Python dependencies and application configuration. But having the large model inside of a container would make the image size and deployment time larger. A more mature design would store model artefacts in a controlled model registry and check the integrity of the stored artefacts before loading.

6.3 SHAP and LIME within the System

The trained model determines the Safe/Phishing classification. Security indicators and SHAP values provide contextual or explanatory information but do not independently override the model's decision. The LIME implementation follows the same separation: the classification result can be tested without the explanation layer, which is the layer responsible for the explanation of the result.

The explainability layer was implemented using SHAP as a post-hoc mechanism. SHAP analyses the behaviour around a prediction to estimate token-level contributions. LIME is model agnostic and creates perturbed versions of the selected email, observes changes in the classifier's probabilities and fits an interpretable local surrogate model. These yield weighted terms for the decision close to that input.

Both explanation methods therefore remain downstream from the trained classifier. SHAP contribution information and LIME influential terms explain model behaviour; they do not produce an independent ground-truth judgement and do not determine whether the email is phishing.

The two implementations also expose different backend configurations. The SHAP implementation uses FastAPI with separate /predict and /explain routes, while the LIME implementation uses Flask and POST /api/analyse. Both keep prediction logic in the backend and allow the browser extension or another client to consume the same underlying classifier without implementing the model in JavaScript.

The computational requirements of explanation are greater than ordinary classification in both implementations. Applying SHAP to DeBERTa-v3-large requires repeated model evaluations, while LIME generates modified versions of the email and continues to ask the classifier. The final project therefore treats explanation as supporting information associated with a prediction rather than as the classification mechanism itself.

6.4 SHAP and LIME Comparison

The trained model determines the Safe/Phishing classification in both implementations. SHAP and LIME operate as post-hoc explanation mechanisms and provide supporting information about the resulting prediction rather than independently determining whether an email is phishing. Although both implementations use the same underlying gated-hybrid classification approach, they differ in their explanation mechanisms, backend configurations and computational workflows.

Aspect	SHAP Implementation	LIME Implementation
Purpose	Explains the Safe/Phishing prediction produced by the trained classifier.	Explains the Safe/Phishing prediction produced by the trained classifier.
Explanation approach	Post-hoc explanation using SHAP.	Post-hoc, model-agnostic explanation using LIME.
How explanation is generated	Analyses model behaviour around the prediction to estimate token-level contributions.	Creates perturbed versions of the selected email, observes changes in classifier probabilities and fits an interpretable local surrogate model.
Explanation output	Token-level contribution information.	Influential terms with signed local weights.
Role in classification	Does not independently determine or override the Safe/Phishing classification.	Does not independently determine or override the Safe/Phishing classification.
Backend framework	FastAPI	Flask
Local backend	http://127.0.0.1:8000	http://127.0.0.1:5000
API routes	/predict for classification and /explain for SHAP explanation.	POST /api/analyse for the analysis workflow.
User interfaces	Streamlit application and browser extension.	Browser extension/API client.
Computational requirement	More computationally expensive than ordinary classification because SHAP requires repeated model evaluations.	More computationally expensive than ordinary classification because LIME generates perturbed emails and repeatedly queries the classifier.
Relationship with classifier	Operates downstream from the trained classifier.	Operates downstream from the trained classifier.

Table 6.1: Comparison of the SHAP and LIME Explainability Implementations

Despite their different explanation mechanisms and backend configurations, both implementations maintain the same fundamental separation between classification and explanation. The trained gated-hybrid model produces the Safe/Phishing prediction,

while SHAP or LIME provides additional information to help interpret that prediction. The explanation output should therefore be treated as supporting information about model behaviour rather than as an independent ground-truth judgement.

6.5 Security and Privacy Considerations

The SHAP and LIME based implementations introduce important security and privacy considerations because email content is transmitted from the user facing interface to a local backend for analysis. Email messages may contain names, email addresses, telephone numbers, account information, business information and other personal, confidential or organizationally sensitive information. Security and privacy are therefore concerned not only with the accuracy of the classifier, but also with how email content, API requests, logs, examples and model outputs are processed.

The two implementations use separate local backend services. The SHAP implementation uses the FastAPI service at <http://127.0.0.1:8000>, while the LIME implementation uses the Flask service at <http://127.0.0.1:5000>. In both cases, communication takes place through the loopback interface during prototype operation. This local arrangement allows the interfaces to communicate with the backend without requiring email content to be transmitted to a remote external service during demonstration. However, the local prototype architecture should not be considered equivalent to a production deployment.

At the API boundary, input validation is important because malformed or excessively large requests should not be passed directly to the machine-learning model or explanation components. In the LIME implementation, malformed JSON, missing text, blank input and excessively large input are rejected, with a maximum input length of 30,000 characters. This provides a basic safeguard against accidentally or intentionally large requests. Such controls are particularly relevant because machine-learning inference requires considerably more computational resources than a typical web request, while explanation generation introduces additional computational cost. LIME repeatedly queries the classifier using perturbed versions of the email, while SHAP also requires repeated model evaluations when calculating token-level contributions.

The input length restriction alone cannot completely prevent resource exhaustion or denial-of-service conditions. Repeated legitimate looking requests could still trigger multiple predictions or explanation operations. If either backend were made accessible beyond the local machine, additional controls would therefore be required, including authentication, authorization, per-client rate limiting, execution timeouts, request queues and concurrent processing limits. Separating ordinary prediction from explanation is particularly useful in SHAP implementation because the more computationally expensive explanation procedure does not need to be executed for every prediction.

The current prototype uses local HTTP communication rather than HTTPS. This is appropriate for the demonstrated loopback configuration but would not be sufficient if either service were deployed remotely. A production deployment would require encrypted HTTPS/TLS communication, server authentication and appropriate access-control mechanisms. Controls would also be required to determine whether submitted email content is temporarily processed, logged or retained. Production logs should avoid storing complete email content, authentication information or other sensitive information unless there is a justified requirement and appropriate protection.

Browser permissions also require careful consideration because the browser extension can operate on webpage content that may contain sensitive email information. Following the principle of least privilege, the extension should request only the permissions required to obtain supported email content and communicate with the appropriate local backend. Restricting extension permissions reduces the potential impact if the extension is compromised (Google 2024). Email content obtained from the browser should not be retained by the extension for longer than required for analysis unless the user has explicitly consented to its storage.

Privacy requirements also apply to the information used during system development, testing and assessment. The Australian Privacy Principles require organizations to take reasonable steps to protect personal information from misuse, interference, loss and unauthorized access, modification or disclosure. Personal information that is no longer required should also be destroyed or de-identified where appropriate and subject to applicable retention requirements (Office of the Australian Information Commissioner 2025). Consequently, real email messages used for testing should be replaced with public benchmark samples where possible or appropriately de-identified. Names, addresses, account numbers, telephone numbers, organizations, message identifiers and confidential links should be replaced with realistic placeholders where their original values are not required.

De-identification should be applied consistently. Removing a person's name may not be sufficient where combinations of information such as job titles, organizations, dates or quoted correspondence could still identify the individual. Screenshots used as project or assessment evidence should therefore also be reviewed separately because browser windows, email headers, terminal output and other interface elements may expose information that is not present in the main email body.

The system also has security limitations at the detection level. The current classifier primarily analyses textual email information and does not independently authenticate sender identity using SPF, DKIM or DMARC results. It does not inspect attachments, resolve shortened URLs, query external threat-intelligence services or comprehensively analyze message-routing headers. Consequently, the SHAP and LIME implementations

should be treated as decision-support mechanisms rather than replacements for established email-security controls.